



Transportation Science

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

The Traveling Salesman Problem with Pickups, Deliveries, and Handling Costs

Maria Battarra, Güneş Erdoğan, Gilbert Laporte, Daniele Vigo,

To cite this article:

Maria Battarra, Güneş Erdoğan, Gilbert Laporte, Daniele Vigo, (2010) The Traveling Salesman Problem with Pickups, Deliveries, and Handling Costs. *Transportation Science* 44(3):383-399. <http://dx.doi.org/10.1287/trsc.1100.0316>

Full terms and conditions of use: <http://pubsonline.informs.org/page/terms-and-conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

Copyright © 2010, INFORMS

Please scroll down for article—it is on subsequent pages



INFORMS is the largest professional society in the world for professionals in the fields of operations research, management science, and analytics.

For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

The Traveling Salesman Problem with Pickups, Deliveries, and Handling Costs

Maria Battarra

Dipartimento di Elettronica, Informatica e Sistemica, University of Bologna, 47521 Cesena (FC), Italy,
maria.battarra@unibo.it

Güneş Erdoğan

Department of Industrial Engineering, Özyeğin University, 34662 Altunizade, İstanbul, Turkey,
gunes.erdogan@ozyegin.edu.tr

Gilbert Laporte

Department of Management Sciences, HEC Montréal, Montréal, Québec J3T 247, Canada,
gilbert.laporte@cirrelt.ca

Daniele Vigo

Dipartimento di Elettronica, Informatica e Sistemica, University of Bologna, 47521 Cesena (FC), Italy,
daniele.vigo@unibo.it

This paper introduces a new variant of the one-to-many-to-one single vehicle pickup and delivery problems (SVPDP) that incorporates the handling cost incurred when rearranging the load at the customer locations. The simultaneous optimization of routing and handling costs is difficult, and the resulting loading patterns are hard to implement in practice. However, this option makes economical sense in contexts where the routing cost dominates the handling cost. We have proposed some simplified policies applicable to such contexts. The first is a two-phase heuristic in which the tour having minimum routing cost is initially determined by optimally solving an SVPDP, and the optimal handling policy is then determined for that tour. In addition, branch-and-cut algorithms based on integer linear programming formulations are proposed, in which routing and handling decisions are simultaneously optimized, but the handling decisions are restricted to three simplified policies. The formulations are strengthened by means of problem specific valid inequalities. The proposed methods have been extensively tested on instances involving up to 25 customers and hundreds of items. Our results show the impact of the handling aspect on the customer sequencing and indicate that the simplified handling policies favorably compare with the optimal one.

Key words: the traveling salesman problem; combinatorial optimization; exact algorithm

History: Received: June 21, 2009; revision received: September 12, 2009; accepted: December 5, 2009. Published online in *Articles in Advance* March 18, 2010.

1. Introduction

Single vehicle pickup and delivery problems (SVPDPs) have received considerable attention in recent years (see Berbeglia et al. 2007). These problems consist of designing an optimal route for a single vehicle making pickups and deliveries to a set of customers. Here we are interested in one-to-many-to-one SVPDPs in which each customer may have a supply and a demand; all demands originate from a depot and all supplies are destined to the depot. A minimum cost circuit for which the vehicle load never exceeds the vehicle capacity has to be determined. One-to-many-to-one SVPDPs have recently been reviewed by Gribkovskaia and Laporte (2008), who have defined two variants: the SVPDP with single demands (SVPDP-P/D), i.e., problems in which customers require only a pickup or only a

delivery, and the SVPDP with combined demands (SVPDP-P&D), i.e., problems in which customers may receive combined pickup and delivery services.

When the vehicle is rear-loaded, the pickup commodities may obstruct the delivery commodities. These obstruction issues have traditionally been taken into account by forcing the tour to have a suitable shape. For example, the traveling salesman problem with backhauls (TSPB) is an SVPDP-P/D in which delivery customers must be served first in the tour, whereas the *double-path* and the *lasso* are solution shapes suitable for the SVPDP-P&D. The double-path SVPDP, proposed by Gribkovskaia et al. (2007), consists of visiting each customer but one twice: the deliveries are first performed, and the pickups are then made by traversing the path in the reverse order. The lasso solution shape, proposed by Halskau and

Løkketangen (1998) and further analyzed by Hoff and Løkketangen (2006), Gribkovskaia et al. (2007), and Hoff et al. (2009), consists of first delivering commodities to a customer subset S , then simultaneously picking up and delivering commodities at each of the remaining customers, and finally visiting the customers of S in the reverse order for deliveries.

These problem variants do not consider any rearrangement of goods in the vehicle. Solution quality can be unsatisfactory whenever the routing cost is dominant with respect to the possible rearrangement cost. In addition, an SVPDP-P&D in which customers do not accept to be visited twice cannot be suitably solved by adopting any of the tour shapes previously presented. Note that the possibility of rearranging goods in the vehicle has not previously been considered in related problems (see, for example, Ladany and Mehrez 1984; Carrabs, Cerulli, and Cordeau 2007a; Carrabs, Cordeau, and Laporte 2007b; Cordeau et al. 2009; Erdoğan, Cordeau, and Laporte 2009; Petersen and Madsen 2009; Petersen, Archetti, and Speranza 2010).

In this paper, we introduce, formulate, and solve a one-to-many-to-one pickup and delivery problem, called the traveling salesman problem with pickups, deliveries, and handling costs (TSPPD-H) in which the handling costs associated with loading and unloading operations are taken into account in the objective function. The TSPPD-H can be defined as follows. Let $G = (V, A)$ be a complete directed graph, where $V = \{0, 1, \dots, n\}$ is the vertex set and A is the arc set. Vertex 0 represents the depot, whereas remaining vertices $V_c = V \setminus \{0\}$ correspond to customers. We define $A_d = \{(0, i) : i \in V_c\}$ and $A_r = \{(i, 0) : i \in V_c\}$ as the sets of arcs having the depot as their tail and head, respectively. Each arc $(i, j) \in A$ has a nonnegative travel time c_{ij} . Customer $i \in V_c$ requires α_i units of commodity a and supplies β_i units of commodity b . Both commodities have the same unit dimensions (for example, boxes or pallets or full and empty containers of the same goods). A vehicle of capacity Q leaves the depot with all delivery requests, visits each customer to possibly deliver commodities of type a and pick up commodities of type b , and returns to the depot with all the pickup requests. Because we considered rear-loaded vehicles, we follow a last-in-first-out (LIFO) loading policy (Pacheco 1997), which we now describe.

Under a LIFO policy, after picking up a type b commodity and loading it at the rear of the vehicle, the unloading of a type a commodity for the next delivery operation is obstructed. In such a case, type b commodities obstructing the delivery should be unloaded first, and reloaded after the delivery of type a commodities. We define an *additional operation* as unloading and reloading one unit of commodity at a customer location. The time required for

one additional operation for commodities a and b is denoted by h_a and h_b , respectively. The optimal solution for the TSPPD-H is then a Hamiltonian circuit on G that minimizes the sum of total travel and additional operation times. The TSPPD-H is $\mathcal{NP}$ -hard because it generalizes the traveling salesman problem (TSP). All applications pertaining to the SVPDP-P/Ds and SVPDP-P&Ds with rear-loading are valid for the TSPPD-H if the vehicle load can be rearranged, and hence a potential for cost reduction is permitted. A specific example arises in the management of bicycle relocations in public bicycle sharing systems. In this application, functioning and defective bicycles are collected from parking locations and often put on a linear rack inside a vehicle. These bicycles are then delivered to other parking locations or to repair centers, respectively.

Note that we have introduced two types of commodities, one for pickup and one for deliveries. However, the models and the algorithms we propose are valid even if each customer is associated with a specific pair of commodities.

The remainder of this paper is organized as follows. In §2, we present an integer linear programming formulation for the TSPPD-H. Next, in §3, given the complexity of the problem, we present some simpler and more tractable policies for the handling problem and we propose integer linear programming formulations based on them. In §4, we describe the components of an exact branch-and-cut algorithm for the problem, as well as a branch-and-cut algorithm based on Benders decomposition. Computational results are presented in §5, followed by conclusions in §6.

2. Integer Linear Programming Formulation for the TSPPD-H

As mentioned in the previous section, at each customer $i \in V_c$, α_i units of commodity a should be unloaded and β_i units of commodity b should be loaded. In the objective function, we do not consider the contribution of these handling operations because the time required to perform them is constant and they are unavoidable. On the other hand, the number of additional unloading and reloading operations depends on the handling decisions made at previously visited customers. Solving the TSPPD-H requires the determination of an optimal sequence of customers and of commodity positions on board on each traversed arc.

Let x_{ij} be a binary variable equal to one if and only if arc $(i, j) \in A$ is in the solution. Numbering the possible loading positions in increasing order, starting from the rear of the vehicle, let the binary variables a_{ij}^k and b_{ij}^k indicate the absence or presence of a type a and b commodity at position $k \in \{1, \dots, Q\}$

within the vehicle, respectively, when the vehicle is traversing arc $(i, j) \in A$. The variables r_i^k indicate whether the commodity at position k is involved in the handling operations at customer i . The last set of variables v_i^k represents the unit handling costs in position k at customer i . We note that because $v_i^k = \sum_{j \in V} (h_a a_{ji}^k + h_b b_{ji}^k) r_i^k$ is not linear, we will appropriately linearize this relation. For notational simplicity, we define $h' = \max\{h_a, h_b\}$. A possible TSPPD-H formulation can be expressed as follows:

(TSPPD-H)

$$\text{minimize} \quad \sum_{(i,j) \in A} c_{ij} x_{ij} + \sum_{i \in V_c} \sum_{k=1}^Q v_i^k - \sum_{i \in V_c} h_a \alpha_i, \quad (1)$$

$$\text{subject to} \quad \sum_{j \in V} x_{ij} = 1, \quad i \in V, \quad (2)$$

$$\sum_{i \in V} x_{ij} = 1, \quad j \in V, \quad (3)$$

$$\sum_{j \in V} \sum_{k=1}^Q (a_{ji}^k - a_{ij}^k) = \alpha_i, \quad i \in V, \quad (4)$$

$$\sum_{j \in V} \sum_{k=1}^Q (b_{ij}^k - b_{ji}^k) = \beta_i, \quad i \in V, \quad (5)$$

$$r_i^k \leq r_i^{k-1}, \quad i \in V_c, k \in \{2, \dots, Q\} \quad (6)$$

$$a_{ij}^k + b_{ij}^k \leq x_{ij}, \quad (i, j) \in A, k \in \{1, \dots, Q\} \quad (7)$$

$$\sum_{j \in V} a_{ij}^k - \sum_{j \in V} a_{ji}^k \leq r_i^k, \quad i \in V_c, k \in \{1, \dots, Q\} \quad (8)$$

$$\sum_{j \in V} a_{ji}^k - \sum_{j \in V} a_{ij}^k \leq r_i^k, \quad i \in V_c, k \in \{1, \dots, Q\} \quad (9)$$

$$\sum_{j \in V} b_{ij}^k - \sum_{j \in V} b_{ji}^k \leq r_i^k, \quad i \in V_c, k \in \{1, \dots, Q\} \quad (10)$$

$$\sum_{j \in V} b_{ji}^k - \sum_{j \in V} b_{ij}^k \leq r_i^k, \quad i \in V_c, k \in \{1, \dots, Q\} \quad (11)$$

$$v_i^k \geq \sum_{j \in V} (h_a a_{ji}^k + h_b b_{ji}^k) - (1 - r_i^k) h', \quad i \in V_c, k \in \{1, \dots, Q\}, \quad (12)$$

$$\sum_{i,j \in S} x_{ij} \leq |S| - 1, \quad S \subset V, |S| \geq 2 \quad (13)$$

$$x_{ij}, a_{ij}^k, b_{ij}^k \in \{0, 1\}, \quad (i, j) \in A, k \in \{1, \dots, Q\}, \quad (14)$$

$$r_i^k \in \{0, 1\}, \quad i \in V_c, k \in \{1, \dots, Q\}, \quad (15)$$

$$v_i^k \geq 0, \quad i \in V_c, k \in \{1, \dots, Q\}. \quad (16)$$

Note that if $x_{ij} = 0$, all the corresponding a_{ij}^k and b_{ij}^k variables are equal to zero. If $x_{ij} = 1$ and a type a commodity is located in position k , then $a_{ij}^k = 1$; if position k is occupied by a type b commodity, then $b_{ij}^k = 1$; if position k is empty, then $a_{ij}^k = b_{ij}^k = 0$. If the commodity in position k is involved in handling operations at customer i , then $r_i^k = 1$; otherwise $r_i^k = 0$, for all $i \in V_c$ and $k \in \{1, \dots, Q\}$.

The objective function (1) calls for the minimization of routing and handling costs, where the constant term $\sum_{i \in V_c} h_a \alpha_i$ is required to obtain the actual cost of the additional handling operations. Constraints (2) and (3) are degree constraints. Constraints (4) and (5) are the flow conservation constraints at each node $i \in V$, for the commodities of type a and b , respectively. Constraints (6) ensure that the commodity in position k at customer i cannot be involved in any handling operation if the commodity in position $k-1$ has not also been involved in a handling operation, as required by the LIFO policy. Constraints (7) define the dependence between the set of variables $a_{ij}^k, b_{ij}^k, x_{ij}$, for all $(i, j) \in A$ and $k \in \{1, \dots, Q\}$. Inequalities (8)–(11) ensure that if no handling operation has been performed at customer i in position k , then the commodity that was occupying position k before servicing customer i has to occupy the same position after the service at this customer. Constraints (12) force the variables v_i^k to assume the proper handling cost value. Finally, constraints (13) are subtour elimination constraints which guarantee the connectivity of the solution.

Although the linear model just presented is valid for the TSPPD-H, it involves a pseudopolynomial number of variables, because their number explicitly depend on the vehicle capacity, which can be arbitrarily large. Moreover the linearizations of nonlinear relations between variables (i.e., constraints (8) to (12)) are likely to weaken the linear relaxation of the model. Consequently, the proposed model can only be used to solve instances with few customers.

3. Handling Policies

We now concentrate on the handling optimization aspect of the problem to derive simplified handling policies which can lead to more tractable formulations.

We first address the problem of finding optimal handling decisions at each customer given a fixed route. This problem can be solved by considering the $O(n^3)$ dynamic programming algorithm proposed by Aslidi (1989, 1991) for the optimal handling of containers into ships. We note that even though this algorithm yields an optimal solution, this solution may involve loading patterns of type a and b commodities that change at every customer, making it very hard to

implement by the driver. Furthermore, the algorithm depends on the sequence of customers to be visited, given as an input data, and is therefore very hard to couple with routing decisions within the framework of an exact algorithm. Nevertheless, one can use this exact handling algorithm to derive a two-phase heuristic for the TSPPD-H. In the first step, an optimal Hamiltonian SVPDP tour is determined through an exact algorithm, and optimal handling decisions are then computed by means of the Aslidis dynamic programming algorithm. The resulting heuristic has a good performance, as shown in §5.

In §3.1, we describe three handling policies that are more suited to practical implementations, and for which we were able to derive integer linear programming formulations. We represent the commodities on board through ordered strings of a and b commodities, in which F and R represent the front and the back of the vehicle, respectively. For example, $(F, a, a, b, b, b, a, a, a, R)$ defines a load pattern in which there are two a commodities, followed by three b commodities and three a commodities at the rear of the vehicle. In §§3.2 and 3.3, we provide integer linear programming formulations based on the application of the policies described in §3.1. Model TSPPD- H_p consists of designing a feasible route for which policy p is applied at each customer.

3.1. Description of the Three Handling Policies

The simplest way of loading the vehicle is to place the picked up commodities at the rear of the vehicle. However, at subsequent customers requiring a delivery, we first have to unload all commodities of type b that are obstructing the unloading of commodities of type a for that customer. After delivering the a commodities, the unloaded b commodities must be reloaded and then the pickup for the customer, if any, is performed. This policy assumes that the commodities on board have the pattern $(F, a, \dots, a, b, \dots, b, R)$. We will refer to this policy as Policy 1.

A second policy consists of unloading all type a commodities at each customer before performing pickup, hence placing all type b commodities at the front of the vehicle, and reloading all remaining type a commodities. This policy assumes that the commodities on board have the pattern $(F, b, \dots, b, a, \dots, a, R)$ and will be referred to as Policy 2.

Note that Policy 1 requires the vehicle to spend some additional time at each customer to unload and reload the type b commodities that are obstructing the delivery of commodity a . On the other hand, the second policy requires time to arrange all the b commodities in a position that is suitable for the delivery of a commodities.

Policy 3 consists of choosing, at each customer, between Policies 1 and 2: the decision maker can opt to just place the type b commodities at the back of the vehicle and unload them at the subsequent customers, or put them at the front by first unloading all type a commodities. Note that for a given tour, determining the best sequence of applications of Policies 1 and 2, is already a difficult problem on its own. Although not necessarily optimal, Policy 3 combines the best features of Policies 1 and 2, while ensuring a reasonably functional handling policy. Policy 3 may include three blocks of commodities: two blocks of type b , one at the front and one at the rear of the vehicle, and a third block of type a in between, i.e., $(F, b, \dots, b, a, \dots, a, b, \dots, b, R)$.

Figure 1 depicts an example comparing the three policies for the case where $h_a = h_b$. The example shows three customers requiring $(3, 4, 3)$ type a commodities (indicated by the lighter commodities on the pallets) and $(2, 4, 4)$ type b commodities (indicated by the darker commodities on the pallets). Observe that Policy 1 requires eight additional operations. Indeed at customer 2 it is necessary to unload two b commodities obstructing the a commodities, whereas at customer 3 the number of b commodities to be unloaded is six. Also note that assuming symmetric distances, route $(1, 2, 3)$ has the same routing cost as route $(3, 2, 1)$, but the number of additional operations required is 12 if Policy 1 is applied to the latter route. Under Policy 2, the number of additional operations to arrange the b commodities in front of the vehicle is 10 (7 operations are required at customer 1, 3 at customer 2). Policy 2 yields a higher handling cost, by requiring a larger number of additional operations compared with Policy 1. Policy 3 produces a better result: by applying the first policy at customer 1 and the second at customer 2, the number of additional operations required decreases to five. More precisely, no additional operations are necessary at customer 1, whereas at customer 2 we first unload the two b commodities to deliver four a commodities, and the three a commodities left on board are then unloaded so that all the b commodities can be arranged in front of the vehicle as required by Policy 1.

Figure 2 provides a comparison between the optimal and the third handling policies for the tour $(1, 2, 3, 4, 5)$. The customers require $(3, 3, 1, 3, 10)$ type a commodities (indicated by the lighter commodities) and $(2, 4, 0, 2, 8)$ type b commodities (indicated by the darker commodities), respectively. At customer 2, the optimal policy inserts the six pickup commodities in an intermediate vehicle position (i.e., after four delivery commodities), whereas Policy 3 inserts them in the front of the vehicle. The overall cost of the optimal handling is 14 and that of Policy 3

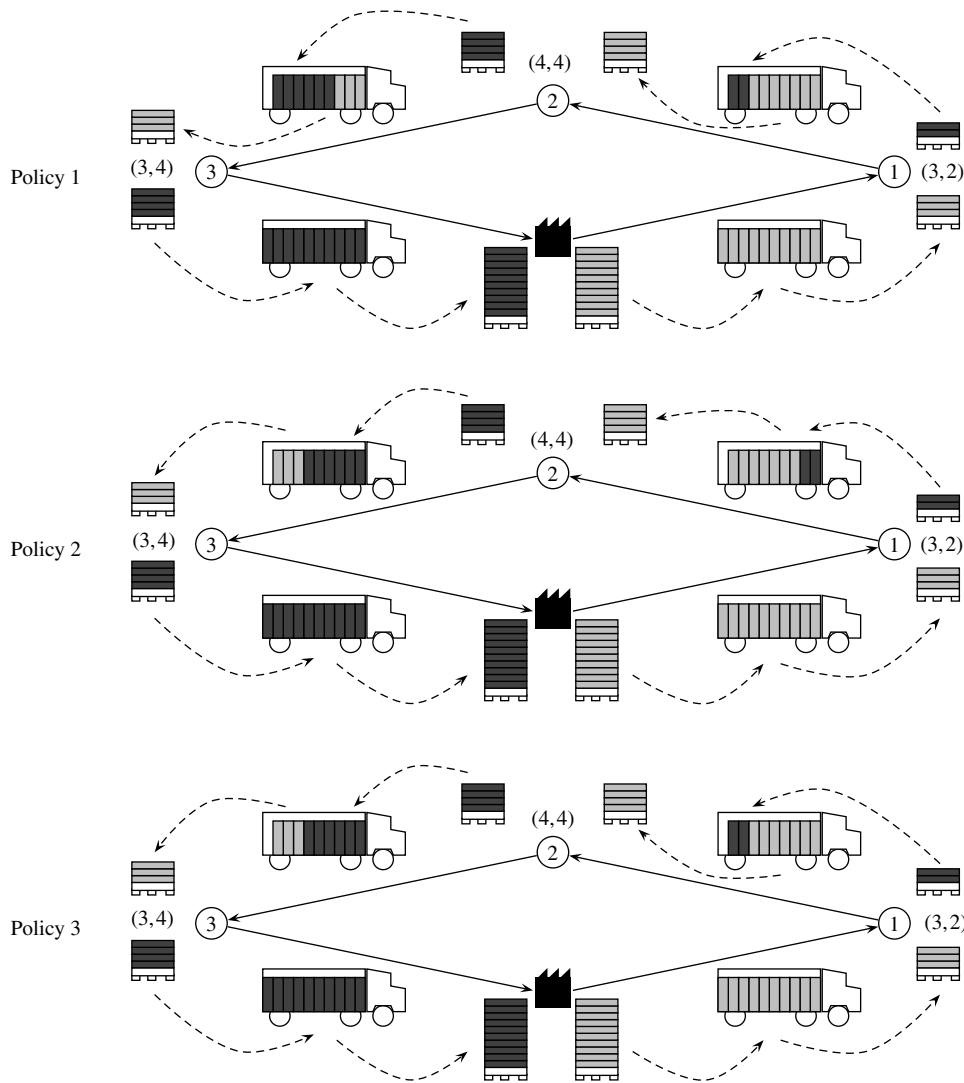


Figure 1 Illustration of the Three Handling Policies

one is 18. We note that even if the optimal policy handling cost is lower than that of Policy 3, the handling instructions for the driver are more complicated by the presence of intermediate insertions.

3.2. Formulations for Policies 1 and 2

The models we present for TSPPD- H_1 and TSPPD- H_2 require two sets of flow variables describing the amount of each commodity on board, in addition to the vehicle flow variables defined in §2. Let y_{ij} and z_{ij} represent the number of commodities of type a and b on board whereas the vehicle traverses arc $(i, j) \in A$, respectively. Considering the first handling policy, our problem can be modeled as follows:

(TSPPD- H_1)

$$\text{minimize} \quad \sum_{(i,j) \in A} c_{ij}x_{ij} + \sum_{(i,j) \in A \setminus A_r, \alpha_j > 0} h_b z_{ij} \quad (17)$$

$$\text{subject to} \quad \sum_{j \in V} x_{ij} = 1, \quad i \in V \quad (18)$$

$$\sum_{i \in V} x_{ij} = 1, \quad j \in V \quad (19)$$

$$\sum_{j \in V} y_{ji} - \sum_{j \in V} y_{ij} \alpha_i, \quad i \in V \quad (20)$$

$$\sum_{j \in V} z_{ij} - \sum_{j \in V} z_{ji} = \beta_i, \quad i \in V \quad (21)$$

$$y_{ij} + z_{ij} \leq Qx_{ij}, \quad (i, j) \in A \quad (22)$$

$$\sum_{i,j \in S} x_{ij} \leq |S| - 1, \quad S \subset V, |S| \geq 2 \quad (23)$$

$$x_{ij} \in \{0, 1\}, \quad (i, j) \in A \quad (24)$$

$$y_{ij}, z_{ij} \geq 0, \quad (i, j) \in A. \quad (25)$$

The objective function incorporates the handling costs by including at each customer location the han-

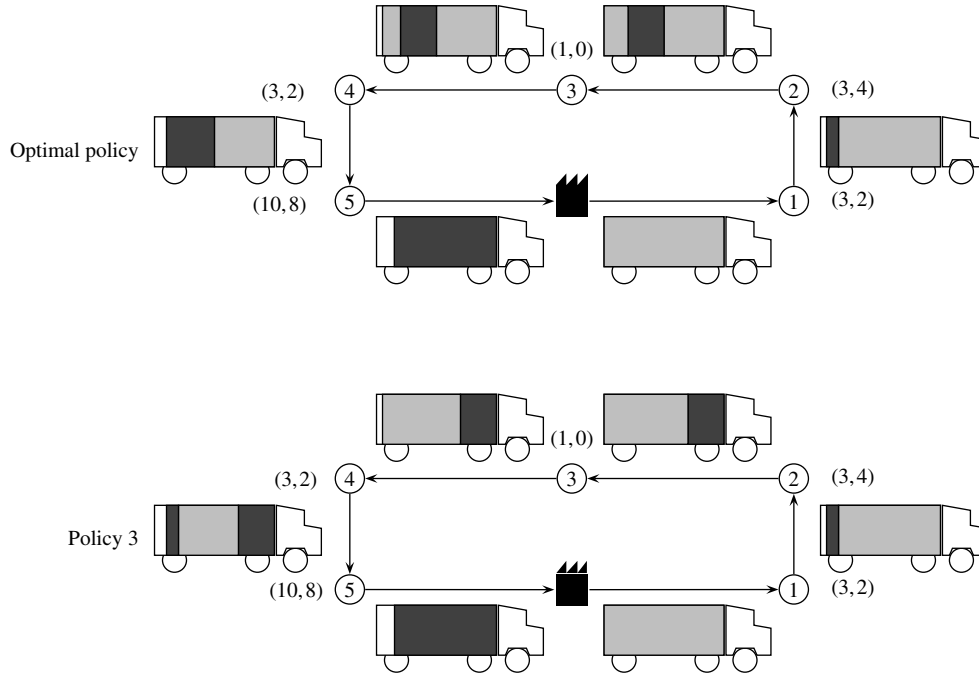


Figure 2 Illustration of the Optimal and the Third Handling Policies

dling costs of the commodities picked up so far. Note that if $\alpha_j = 0$, no extra handling is needed at customer j . Constraints (18) and (19) are the vehicle flow conservation constraints, whereas the constraints (20), (21) are the flow conservation constraints for commodities of type a and b . Constraints (22) enforce the capacity restriction, and constraints (23) are the usual subtour elimination constraints.

To model Policy 2, we modify the objective function of the model presented for Policy 1. In fact for this formulation, we consider that the handling cost is incurred because of the type a commodities on board instead of type b . Note that if $\beta_i = 0$ we do not need to pay any reloading cost. The resulting formulation is:

(TSPPD- H_2)

$$\text{minimize} \quad \sum_{(i,j) \in A} c_{ij}x_{ij} + \sum_{(i,j) \in A \setminus A_d, \beta_i > 0} h_a y_{ij} \quad (26)$$

$$\text{subject to} \quad (18) \text{ to } (25). \quad (27)$$

We now prove that we can use the solution of the TSPPD- H_1 to construct a solution for the TSPPD- H_2 when the distance matrix is symmetric.

PROPOSITION 1. *Consider an instance of the TSPPD- H_1 and construct an instance of TSPPD- H_2 by permuting the values of α_i and β_i , for all $i \in V_c$. Given a feasible solution for TSPPD- H_1 , traversing it in the reverse order yields a feasible solution for TSPPD- H_2 .*

PROOF. For the sake of simplicity, assume that the vertices are renumbered according to the sequence

in which they appear in the feasible solution, and hence the route is $(0, 1, 2, \dots, n-1, n, 0)$. Because this solution is feasible, the load on each arc belonging to the solution does not exceed the vehicle capacity. In particular on arc $(0, 1)$ the load is $\sum_{i=1}^n \alpha_i$, on arc $(1, 2)$ the load is $\beta_1 + \sum_{i=2}^n \alpha_i$, and so on, up to the last two arcs $(n-1, n)$ and $(n, 0)$, on which the load is $\sum_{i=1}^{n-1} \beta_i + \alpha_n$ and $\sum_{i=1}^n \beta_i$, respectively. If the route is traversed in the reverse order, i.e., $(0, n, n-1, \dots, 2, 1, 0)$, and we reverse the roles of α_i and β_i , for all $i \in V_c$, it is easy to verify that the load on each arc of the reversed circuit is equal to that of the arc in the corresponding position of the original route. Because these loads never exceed the vehicle capacity the reversed circuit is also feasible. $\square$

Note that this is not necessarily true if we do not reverse the roles of α_i and β_i , for all $i \in V_c$. Also, if the travel time matrix is symmetric, the routing cost of the original and reversed circuits are equal. Given a feasible circuit, say $(0, 1, 2, \dots, n-1, n, 0)$ in which $\alpha_i > 0$, $i \in V_c$, we now compute the total handling cost associated with it under Policy 1 as

$$h_b \sum_{i=1}^{n-1} (n-i) \beta_i = h_b \sum_{i=1}^n (n-i) \beta_i, \quad (28)$$

whereas under Policy 2, if $\beta_i > 0$, $i \in V_c$, the handling cost is

$$h_a \sum_{i=2}^n (i-1) \alpha_i = h_a \sum_{i=1}^n (i-1) \alpha_i. \quad (29)$$

When the circuit is traversed in reverse order, i.e., $(0, n, n-1, \dots, 2, 1, 0)$ and $\beta_i > 0$, $i \in V_c$, we can easily

compute the handling cost associated with Policy 1 by defining $j = n - i + 1$ and by considering $h_b^* = h_a$, $\beta_j^* = \alpha_{n-i+1}$, and $\alpha_j^* = \beta_{n-i+1}$, hence obtaining

$$h_b^* \sum_{j=1}^n (n-j) \beta_j^* = h_a \sum_{i=1}^n (i-1) \alpha_i. \quad (30)$$

We can now state the following result on the equivalence between Policies 1 and 2.

PROPOSITION 2. *Given any instance of TSPPD-H with a symmetric distance matrix and $\beta_i > 0$, $i \in V_c$, an optimal solution for TSPPD-H₂ can be obtained by reversing the order of an optimal solution for TSPPD-H₁ for an instance in which α_i and β_i , for all $i \in V_c$, and h_a and h_b are permuted.*

PROOF. Given an optimal solution obtained with Policy 1, from Proposition 1 we know that the reversed solution is also feasible for the original instance. The routing cost is the same because the travel time matrix is symmetric. Finally, Equations (28), (29), and (30) show that the handling cost is equal.

3.3. Policy 3

Let s_i be equal to one if Policy 1 is applied at customer i , and zero if Policy 2 is adopted. For each $(i, j) \in A$, let the number of commodities of type b arranged at the front of the vehicle be denoted by w_{ij} and those at the rear be denoted by z_{ij} . In addition, we introduce variables $p_i, q_i \geq 0$, for all $i \in V_c$. The resulting formulation can be expressed as follows:

(TSPPD-H₃)

$$\begin{aligned} \text{minimize} \quad & \sum_{(i,j) \in A} c_{ij} x_{ij} + \sum_{(i,j) \in A \setminus A_r, \alpha_j > 0} h_b z_{ij} \\ & + \sum_{i \in V_c} q_i \end{aligned} \quad (31)$$

$$\text{subject to} \quad \sum_{j \in V} x_{ij} = 1, \quad i \in V \quad (32)$$

$$\sum_{i \in V} x_{ij} = 1, \quad j \in V \quad (33)$$

$$\sum_{j \in V} y_{ji} - \sum_{j \in V} y_{ij} = \alpha_i, \quad i \in V_c \quad (34)$$

$$\begin{aligned} \sum_{j \in V} (w_{ij} + z_{ij}) - \sum_{j \in V} (w_{ji} + z_{ji}) &= \beta_i, \\ i &\in V_c \end{aligned} \quad (35)$$

$$\begin{aligned} \sum_{j \in V} w_{ij} - \sum_{j \in V} w_{ji} &\leq (1 - s_i) \left(\sum_{j \in V_c} \beta_j \right), \\ i &\in V_c \end{aligned} \quad (36)$$

$$w_{ij} + y_{ij} + z_{ij} \leq Q x_{ij}, \quad (i, j) \in A \quad (37)$$

$$\sum_{i, j \in S} x_{ij} \leq |S| - 1, \quad S \subset V, |S| \geq 2 \quad (38)$$

$$\begin{aligned} \sum_{j \in V} h_a y_{ij} + \sum_{j \in V} h_b z_{ji} &= p_i + q_i, \\ i &\in V_c: \alpha_i = 0 \end{aligned} \quad (39)$$

$$\begin{aligned} p_i &\leq \left(h_a \sum_{j \in V_c} \alpha_j + h_b \sum_{j \in V_c} \beta_j \right) s_i, \\ i &\in V_c: \alpha_i = 0 \end{aligned} \quad (40)$$

$$\begin{aligned} q_i &\leq \left(h_a \sum_{j \in V_c} \alpha_j + h_b \sum_{j \in V_c} \beta_j \right) (1 - s_i), \\ i &\in V_c: \alpha_i = 0 \end{aligned} \quad (41)$$

$$\sum_{j \in V} h_a y_{ij} = p_i + q_i, \quad i \in V_c: \alpha_i > 0 \quad (42)$$

$$p_i \leq \left(h_a \sum_{j \in V_c} \alpha_j \right) s_i, \quad i \in V_c: \alpha_i > 0 \quad (43)$$

$$\begin{aligned} q_i &\leq \left(h_a \sum_{j \in V_c} \alpha_j \right) (1 - s_i), \\ i &\in V_c: \alpha_i > 0 \end{aligned} \quad (44)$$

$$x_{ij} \in \{0, 1\}, \quad (i, j) \in A \quad (45)$$

$$s_i \in \{0, 1\}, \quad i \in V_c \quad (46)$$

$$y_{ij}, w_{ij}, z_{ij} \geq 0, \quad (i, j) \in A \quad (47)$$

$$p_i, q_i \geq 0, \quad i \in V_c. \quad (48)$$

The objective function (31) minimizes the sum of routing and handling costs. The term $\sum_{(i,j) \in A \setminus A_r, \alpha_j > 0} h_b z_{ij}$ is the total handling cost under Policy 1. The term $\sum_{i \in V_c} q_i$ adjusts the handling costs to consider the cases in which Policy 2 is applied. More precisely, constraints (39)–(41), with $\alpha_i = 0$, set the appropriate q_i handling cost for customers without delivery demand: if the first policy is applied, the handling cost is $q_i = 0$, whereas if the second policy is applied $q_i = \sum_{j \in V} h_a y_{ij} + \sum_{j \in V} h_b z_{ji}$. Constraints (42)–(44), with $\alpha_i > 0$, set the appropriate q_i handling cost for customers with delivery demand: if Policy 1 is applied $q_i = 0$, whereas if Policy 2 is applied $q_i = \sum_{j \in V} h_a y_{ij}$. Note that, if $\alpha_i > 0$, $\beta_i = 0$ and $\sum_{j \in V} z_{ji} = 0$, Policy 2 is never convenient.

Constraints (32) and (33) are degree constraints, whereas the constraints (34)–(36) are flow conservation constraints for the a commodities, the b commodities and for the b commodities at the front of the vehicle w_{ij} , respectively. Constraints (37) are the capacity constraints, whereas (38) are subtour elimination constraints. Constraints (45)–(48) define the nature of the variables.

4. Exact Algorithms

The formulations TSPPD-H₁, TSPPD-H₂, and TSPPD-H₃ can be solved to optimality by branch-and-cut or through a Benders-based branch-and-cut

algorithm. In what follows, we first describe some shared features of both methods, namely the lower bounds (see §4.1) and the valid inequalities and related separation procedures (see §4.2). We then present a summary of the standard branch-and-cut algorithm (see §4.3) followed by a description of the Benders-based branch-and-cut algorithm (see §4.4).

4.1. Lower Bounds

In the following, we present lower bounds on the handling cost for each of the simplified handling policies. Given a handling policy p , the corresponding handling cost lower bound is LB_p . In this subsection, we assume that the customers are numbered in the order the vehicle visits them, for the sake of simplicity.

We first focus on LB_1 . Disregarding the routing costs associated with the TSP, the optimal tours are those minimizing the number of additional handling operations. These tours respect certain deterministic characteristics, for example the first customers in the tour are those with no pickup (i.e., $i \in V_c: \beta_i = 0$), and the last customers are those with no delivery (i.e., $i \in V_c: \alpha_i = 0$). The handling cost contribution of these two customer sets is then zero. The remaining set of customers, say $C = \{i \in V_c: \alpha_i \neq 0, \beta_i \neq 0\}$, occupies the intermediate tour positions and customers are ordered by increasing pickup value, i.e., $s_i, i \in C, \beta_i \leq \beta_{i+1}$. Tours respecting these characteristics minimize the number of additional operations under Policy 1. Their handling cost LB_1 can be computed as

$$LB_1 = \sum_{i=1}^{|C|} h_b \beta_i (|C| - i). \quad (49)$$

Concerning Policy 2, a similar idea is applied. The first and last positions of the tour are still occupied by the customers with no pickup and no delivery, respectively. The intermediate customers are ordered in decreasing delivery value, so as to reduce as much as possible the amount of a commodities obstructing the rearrangement of goods in the vehicle. Considering such kinds of tours, LB_2 can be computed as

$$LB_2 = \sum_{i=1}^{|C|} h_a \alpha_i (i - 1). \quad (50)$$

Under Policy 3, the customers in the set C cannot be ordered by increasing or decreasing pickup or delivery values, but a minimum handling cost can be computed for each of them. The minimum handling cost incurred at each customer in C , with the exception of the last one served, is

$$\min\{h_a \bar{a}, h_b \beta_i\}, \quad (51)$$

where $\bar{a} = \min_{i \in C} \{\alpha_i\}$. In fact, if Policy 1 is applied, at least the β_i commodities have to be unloaded at

the next customer; otherwise, if Policy 2 is applied, at least the $\bar{a}$ type a commodities of the customer in C with the smallest type a demand have to be unloaded. The resulting lower bound can then be computed as

$$LB_3 = \left(\sum_{i \in C} \min\{h_a \bar{a}, h_b \beta_i\} - \max_{i \in C} \{h_a \bar{a}, h_b \beta_i\} \right). \quad (52)$$

Note that we deducted from LB_3 the quantity $\max_{i \in C} \{h_a \bar{a}, h_b \beta_i\}$, which is an overestimation of the handling cost for the last customer visited in C .

4.2. Valid Inequalities and Related Separation Procedures

The subtour elimination constraints (23) with $|S| \geq 3$ are dynamically generated as cutting planes and included in our models after applying the exact separation procedure based on the *Maximum Flow* algorithm of Edmonds and Karp (1972) (see Padberg and Grötschel 1985). However, these constraints defined with $|S| = 2$ are included in the initial relaxation given that their number is relatively small.

Other cuts are also generated. Capacity constraints (37) can be strengthened as follows:

$$y_{ij} + z_{ij} + w_{ij} \leq x_{ij} (Q - \min\{0, \alpha_i - \beta_i, -\alpha_i + \beta_i\}). \quad (53)$$

Constraints (53) are valid inequalities for TSPPD- H_3 , but they can be adapted to TSPPD- H_1 and TSPPD- H_2 by omitting the w_{ij} variables. Note that if $\alpha_i = \beta_i$, for all $i \in V_c$, then the total flow on each arc is constant. In this case constraint (53) becomes:

$$y_{ij} + z_{ij} + w_{ij} = x_{ij} \sum_{i \in V_c} \alpha_i = x_{ij} \sum_{i \in V_c} \beta_i. \quad (54)$$

In addition, we can introduce the constraints

$$\sum_{i \in S, j \in V \setminus S} x_{ij} \geq \left\lceil \frac{\sum_{i \in S} (\beta_i - \alpha_i)}{Q} \right\rceil, \quad S \subset V: |S| > 1 \quad (55)$$

which have been dynamically included in our model by again applying the exact separation procedure based on the *Maximum Flow* algorithm of Edmonds and Karp (1972).

Note that all inequalities valid for the TSP can be applied to the x_{ij} variables of the three formulations (see Naddef and Thienel 2002, Fischetti and Toth 1997). However, in our preliminary computational tests, these inequalities did not yield significant improvements, and were not used in further tests.

The last inequalities considered force the handling costs to be higher than the lower bounds LB_1 and LB_2 for Policies 1 and 2, respectively. In fact, given the valid lower bound on the amount of additional handling operations required for Policy 1 presented in §4.1, we can impose

$$\sum_{(i,j) \in A \setminus (A_r \cup A_d)} z_{ij} \geq LB_1, \quad (56)$$

and analogously for Policy 2:

$$\sum_{(i,j) \in A \setminus (A_r \cup A_d)} y_{ij} \geq LB_2. \quad (57)$$

Similar inequality can be also derived for Policy 3. However, it has not been included in our algorithm because LB_3 is not generally a tight lower bound.

4.3. Summary of the Branch-and-Cut Algorithm

We now summarize our branch-and-cut algorithm which is applicable to any policy p .

Step 0 (Preprocessing). Delete from G the arcs $(0, i)$, $i \in V_c$ such that $\sum_{j \in V_c} \alpha_j + \beta_i - \alpha_i > Q$.

Step 1 (Root node). Set the cost z^* of the best known solution equal to infinity. Solve a relaxation of formulation TSPPD- H_p , in which subtour elimination constraints and integrality constraints are omitted. Insert this problem in a list.

Step 2 (Node selection). If the list is empty, stop. Else select and remove a problem from the list according to a best-first criterion.

Step 3 (Subproblem solution). Solve the current problem. Let z be the objective function value. If $z \geq z^*$, go to Step 2.

Step 4 (Constraint generation). Generate all identified violated subtour elimination constraints (38) and capacity constraints (see §4.2), and add them to the current problem. If at least one violated constraint is generated, go to Step 3.

Step 5 (Integrality check). If the solution is fractional go to Step 6; else set $z^* = z$, and go to Step 2.

Step 6 (Branching). Construct two new problems by branching on a binary fractional variable. Add the two new problems to the list and go to Step 2.

In our experiments, we have used CPLEX as an LP solver, and Steps 2, 3, 5, 6 were handled by the CPLEX branch-and-cut framework.

4.4. Benders-Based Branch-and-Cut Algorithm

The three models TSPPD- H_1 , TSPPD- H_2 , and TSPPD- H_3 were solved by branch-and-cut. This approach proved capable of solving moderate sized instances. However, we have further investigated alternative resolution techniques for the three simplified policies, by developing a Benders decomposition based branch-and-cut algorithm (see Benders 1962).

Benders decomposition splits the models TSPPD- H_1 , TSPPD- H_2 , and TSPPD- H_3 into two parts. The first, called the *master problem*, uses the integer variables and the second one, called the *subproblem*, uses the flow variables and the associated constraints. In what follows, we describe the master problem and the subproblem (i.e., the primal version) for Policy 3. The formulations TSPPD- H_1 and TSPPD- H_2 can be decomposed in a similar way.

(Master₃)

$$\text{minimize} \quad \sum_{(i,j) \in A} c_{ij} x_{ij} + u \quad (58)$$

$$\text{subject to} \quad \sum_{j \in V} x_{ij} = 1, \quad i \in V \quad (59)$$

$$\sum_{i \in V} x_{ij} = 1, \quad j \in V \quad (60)$$

$$\sum_{i,j \in S} x_{ij} \leq |S| - 1, \quad S \subset V, |S| \geq 2 \quad (61)$$

$$x_{ij} \in \{0, 1\}, \quad (i, j) \in A \quad (62)$$

$$s_i \in \{0, 1\}, \quad i \in V_c \quad (63)$$

$$u \geq 0. \quad (64)$$

(SubPb₃)

$$\text{minimize} \quad \sum_{i \in V} \sum_{j \in V_c: \alpha_j > 0} h_b z_{ij} + \sum_{i \in V_c} q_i \quad (65)$$

$$\text{subject to} \quad \sum_{j \in V} y_{ji} - \sum_{j \in V} y_{ij} = \alpha_i, \quad i \in V \quad (66)$$

$$\sum_{j \in V} (w_{ij} + z_{ij}) - \sum_{j \in V} (w_{ji} + z_{ji}) = \beta_i, \quad i \in V \quad (67)$$

$$\sum_{j \in V} w_{ij} - \sum_{j \in V} w_{ji} \leq (1 - s_i^*) \sum_{j \in V_c} \beta_j, \quad i \in V \quad (68)$$

$$w_{ij} + y_{ij} + z_{ij} \leq Q x_{ij}^*, \quad (i, j) \in A \quad (69)$$

$$\sum_{j \in V} h_a y_{ij} + \sum_{j \in V} h_b z_{ji} = p_i + q_i, \quad i \in V_c: \alpha_i = 0 \quad (70)$$

$$p_i \leq \left(h_a \sum_{j \in V_c} \alpha_j + h_b \sum_{j \in V_c} \beta_j \right) s_i^*, \quad i \in V_c: \alpha_i = 0 \quad (71)$$

$$q_i \leq \left(h_a \sum_{j \in V_c} \alpha_j + h_b \sum_{j \in V_c} \beta_j \right) (1 - s_i^*), \quad i \in V_c: \alpha_i = 0 \quad (72)$$

$$\sum_{j \in V} h_a y_{ij} = p_i + q_i, \quad i \in V_c: \alpha_i > 0 \quad (73)$$

$$p_i \leq \left(h_a \sum_{j \in V_c} \alpha_j \right) s_i^*, \quad i \in V_c: \alpha_i > 0 \quad (74)$$

$$q_i \leq \left(h_a \sum_{j \in V_c} \alpha_j \right) (1 - s_i^*), \quad i \in V_c: \alpha_i > 0 \quad (75)$$

$$w_{ij}, y_{ij}, z_{ij} \geq 0, \quad (i, j) \in A \quad (76)$$

$$p_i, q_i \geq 0, \quad \forall i \in V_c. \quad (77)$$

The master problem and the subproblem are solved at each iteration through a Benders decomposition approach. The master problem deals with the integer variables x_{ij} and s_i , whereas variable u represents the handling cost. The subproblem determines the current u value by considering the commodity flow variables w_{ij} , z_{ij} , y_{ij} , once the values x_{ij}^* and s_i^* are provided by the master problem.

The following constraints, called *Benders inequalities*, are valid for the master problem:

$$u \geq \sum_{(i,j) \in A} \pi_{ij} x_{ij} + \sum_{i \in V_c} \mu_i s_i + \rho, \quad (78)$$

where π , μ , ρ are the coefficients obtained from the optimal dual variables of the subproblem. It is even possible to include cuts when the subproblem produces infeasible solutions, by considering infeasibility cuts derived from the extreme rays of the subproblem dual polyhedron. However, in our experimentation, we have observed that the infeasibility cuts returned by the dual subproblem are very weak. Hence, we have discarded these inequalities, given that the capacity constraints (55) were sufficient to guarantee the solution feasibility.

We have upgraded (78) into *Pareto-optimal* inequalities, by solving a second stage subproblem with an interior point (x', s') used in place of (x^*, s^*) , and including the optimal objective function value of the first stage as a constraint (we refer to Magnanti and Wong 1981 for a comprehensive illustration). As the choice of the interior point, we have started with $x'_{ij} = 1/(|V| - 1)$, $(i, j) \in A$, and $s_i = 0.5$ for all $i \in V_c$. At the subsequent iterations, however, we have replaced x' by $\lambda x' + (1 - \lambda)x^*$, with $\lambda = 0.95$. This approach helped us to build a memory of the fractional solutions encountered into the interior point, and decreased the total number of Pareto-optimal inequalities as well as the CPU time requirement.

The Benders-based branch-and-cut algorithm can be summarized by slightly modifying the scheme described in §4.3. The differences are in Step 1, in which we solve a master problem analogous relaxation (i.e., subtour and integrality constraints are omitted), and in Step 4, in which Benders inequalities are included as a last set, if any is violated.

5. Computational Results

The problem introduced in this paper and the recommended operating policies make sense in contexts where the routing cost dominates the handling cost. We have therefore tested our algorithms on instances exhibiting this characteristic.

In our computational experiments, we have used an adaptation to TSPPD-H of the instances proposed by Gendreau, Laporte, and Vigo (1999) for the SVPDP-P&D (available online at <http://webpages.ull.es/>

users/hhperez/PDsite/index.html). More precisely, we have considered the 10 instances with 50 customers from the *Euclidean* set and, for each of them, we have extracted smaller instances by considering the first 5, 10, 15, 20, and 25 customers, respectively. The number of customers is denoted in the tables as $|V_c|$, and instances with the same number of customers are distinguished through an identification number $\text{Id} = 1, \dots, 10$.

In the original instances, the pickup and delivery values were randomly generated according to a discrete uniform distribution on $[0, 100]$. To keep the handling component moderate, we have reduced their values as follows. Denoting as p_i , $i \in V_c$, the pickup value of customer $i = 1, \dots, |V_c|$ in the original SVPDP-P&D instance, we scaled it down to $p'_i = \max\{1, p_i \bmod 20\}$ and we subdivided this quantity between α_i and β_i as

$$\beta_i = \left\lfloor p'_i \frac{i \bmod 5}{5} \right\rfloor \quad \text{and} \quad \alpha_i = p'_i - \beta_i. \quad (79)$$

We have defined the vehicle capacity as $Q = \max\{\sum_{i \in V_c} \alpha_i, \sum_{i \in V_c} \beta_i\}$. As for the handling cost, we have only considered the case $h_a = h_b = h$ and we have used $h \in \{0.1, 0.5, 1\}$. These values produced appropriate handling cost with respect to the routing ones, according to our extensive testing. Moreover, instances in which the handling cost is comparable to the routing cost may be better solved by considering alternative route shapes. Therefore, we have limited our test bed by excluding instances with $h = 0.5$ and $|V_c| = 25$ and instances with $h = 1$ and $|V_c| \geq 20$, in which the handling cost turned out to be too large.

Our algorithms were coded in C and run on an AMD Athlon 64 × 2 Dual, 2.20 GHz PC, using CPLEX 11.2 as integer linear programming solver. Table 1 presents the results obtained by considering the simple two-phase heuristic described in §3. We have constructed an optimal Hamiltonian SVPDP tour by solving model TSPPD-H₁ with $h_a = h_b = 0$. The routing cost of this tour is denoted by z_R . Given this tour, we have computed the handling cost by applying Policies 1–3 and the optimal cost as described in Aslidis (1989). The corresponding handling costs are denoted by z_H^1 , z_H^2 , z_H^3 , and z_H^* , respectively.

Even considering the smaller h coefficient (i.e., $h = 0.1$), the table shows the importance of handling policies: the average handling cost obtained by applying the optimal handling policy is 12.7 (3.7% of the average routing cost), but it is 40.8 and 52.2 under Policies 1 and 2, respectively (11.8% and 15.1% of the average routing cost). By using the first two straightforward policies, the handling costs is up to six times larger than the optimal one. Note also that Policy 3 guarantees an almost perfect performance (i.e., 4.7% of the average routing costs) and is easier to imple-

Table 1 Solution Values for the Three Handling Policies and the Optimal Policy by Considering the Two-Phase Algorithm

$ V_c $	Id	Routing	Handling ($h = 0.1$)				z_H^*/z_H		
		z_R	z_H^1	z_H^2	z_H^3	z_H^*	$h = 0.1$ (%)	$h = 0.5$ (%)	$h = 1$ (%)
5	1	295	2.0	4.4	1.5	1.5	0.5	2.5	5.1
5	2	260	2.7	2.4	1.5	1.5	0.6	2.9	5.8
5	3	279	1.9	1.1	0.8	0.8	0.3	1.4	2.9
5	4	252	2.0	2.2	0.7	0.7	0.3	1.4	2.8
5	5	196	5.5	6.9	2.7	2.7	1.4	6.9	13.8
5	6	176	5.5	4.6	2.6	2.6	1.5	7.4	14.8
5	7	233	4.0	3.4	2.2	2.2	0.9	4.7	9.4
5	8	191	1.8	1.0	0.9	0.9	0.5	2.4	4.7
5	9	276	1.8	4.0	1.3	1.3	0.5	2.4	4.7
5	10	187	2.3	6.7	2.3	1.9	1.0	5.1	10.2
10	1	333	16.7	17.6	7.8	5.9	1.8	8.9	17.7
10	2	320	14.9	16.4	6.2	5.3	1.7	8.3	16.6
10	3	338	7.3	8.7	3.1	3.1	0.9	4.6	9.2
10	4	299	9.2	10.2	5.0	4.3	1.4	7.2	14.4
10	5	315	12.2	16.2	6.3	5.9	1.9	9.4	18.7
10	6	245	17.0	19.1	7.7	6.2	2.5	12.7	25.3
10	7	285	17.9	19.2	8.3	7.7	2.7	13.5	27.0
10	8	315	8.0	16.4	5.7	5.3	1.7	8.4	16.8
10	9	283	11.2	18.7	6.9	5.9	2.1	10.4	20.8
10	10	277	10.0	16.1	4.6	4.4	1.6	7.9	15.9
15	1	367	29.4	32.4	15.6	11.3	3.1	15.4	30.8
15	2	368	44.8	49.7	14.2	13.0	3.5	17.7	35.3
15	3	400	15.5	23.0	6.4	6.2	1.6	7.8	15.5
15	4	364	28.7	24.2	11.7	9.8	2.7	13.5	26.9
15	5	390	39.1	40.2	15.6	12.4	3.2	15.9	31.8
15	6	335	31.4	29.1	11.8	10.1	3.0	15.1	30.1
15	7	360	46.9	67.3	22.4	17.6	4.9	24.4	48.9
15	8	380	21.4	39.4	13.9	11.7	3.1	15.4	30.8
15	9	334	30.0	56.9	17.2	13.8	4.1	20.7	41.3
15	10	344	30.6	57.7	16.6	13.4	3.9	19.5	39.0
20	1	419	49.4	69.5	25.0	17.9	4.3	21.4	
20	2	401	85.2	87.8	26.7	21.4	5.3	26.7	
20	3	431	36.8	41.1	12.1	11.2	2.6	13.0	
20	4	401	67.7	59.4	19.9	17.2	4.3	21.4	
20	5	406	72.7	87.3	30.8	23.6	5.8	29.1	
20	6	359	64.4	65.7	22.4	16.8	4.7	23.4	
20	7	405	78.0	100.2	30.6	21.8	5.4	26.9	
20	8	400	41.4	78.3	22.7	17.2	4.3	21.5	
20	9	367	59.2	83.4	24.5	19.7	5.4	26.8	
20	10	411	60.0	108.4	27.3	20.7	5.0	25.2	
25	1	460	99.3	139.6	40.0	28.3	6.2		
25	2	464	117.7	128.5	32.5	26.2	5.6		
25	3	464	58.7	84.3	23.9	18.6	4.0		
25	4	449	101.6	107.2	29.1	24.6	5.5		
25	5	421	126.6	143.1	47.4	33.6	8.0		
25	6	381	82.8	94.8	29.6	20.7	5.4		
25	7	426	112.3	133.7	39.2	28.8	6.8		
25	8	422	65.6	131.4	31.9	25.0	5.9		
25	9	384	94.0	114.4	37.0	28.2	7.3		
25	10	426	94.8	136.8	36.8	25.9	6.1		
Avg.		345.9	40.8	52.2	16.3	12.7	3.3	13.2	19.6

Table 2 Solution Values and Computing Times for the Three Handling Policies by Considering the Branch-and-Cut Algorithm and $h = 0.1$

$ V_c $	Id	Two-phase		Policy 1			Policy 2			Policy 3		
		z^*	z_H^*	z^*	z_H^1	Seconds	z^*	z_H^2	Seconds	z^*	z_H^3	Seconds
5	1	296.5	1.5	297.0	2.0	0	299.4	4.4	0	296.5	1.5	0
5	2	261.5	1.5	262.7	2.7	0	262.4	2.4	0	261.5	1.5	0
5	3	279.8	0.8	280.9	1.9	0	280.1	1.1	0	279.8	0.8	0
5	4	252.7	0.7	254.0	2.0	0	254.2	2.2	0	252.7	0.7	0
5	5	197.3	2.7	197.7	1.7	0	200.4	4.4	0	197.3	1.3	0
5	6	178.6	2.6	179.3	3.3	0	180.6	4.6	0	178.6	2.6	0
5	7	235.2	2.2	237.0	4.0	0	236.4	3.4	0	235.2	2.2	0
5	8	191.6	0.9	192.5	1.5	1	191.9	0.9	0	191.6	0.6	0
5	9	277.3	1.3	277.8	1.8	0	280.0	4.0	0	277.3	1.3	0
5	10	188.7	1.9	189.3	2.3	0	190.4	3.4	0	189.0	2.0	0
10	1	338.9	5.9	349.7	16.7	1	350.6	17.6	0	340.8	7.8	1
10	2	325.3	5.3	334.9	14.9	0	336.4	16.4	0	326.2	6.2	0
10	3	341.1	3.1	345.3	7.3	1	346.7	8.7	0	341.1	3.1	0
10	4	303.3	4.3	308.2	9.2	2	306.8	7.8	1	304.0	5.0	1
10	5	320.9	5.9	327.2	12.2	0	331.2	16.2	0	321.3	6.3	1
10	6	251.2	6.2	262.0	17.0	2	264.1	19.1	1	252.7	7.7	0
10	7	292.7	7.7	302.9	17.9	1	304.2	19.2	0	293.3	8.3	1
10	8	320.3	5.3	323.0	8.0	0	331.4	16.4	0	320.7	5.7	0
10	9	288.9	5.9	294.2	11.2	0	301.7	18.7	1	289.9	6.9	0
10	10	281.4	4.4	287.0	10.0	0	293.1	16.1	1	281.6	4.6	0
15	1	378.3	11.3	395.8	28.8	4	399.4	32.4	2	382.0	15.0	4
15	2	381.0	13.0	409.4	37.4	13	417.4	45.4	12	382.2	14.2	19
15	3	406.2	6.2	415.5	15.5	2	423.0	23.0	1	406.4	6.4	1
15	4	373.8	9.8	390.3	25.3	7	387.5	22.5	5	375.5	10.5	50
15	5	402.4	12.4	426.8	30.8	4	429.2	33.2	5	405.6	15.6	4
15	6	345.1	10.1	365.4	30.4	5	363.3	28.3	6	346.7	11.7	2
15	7	374.1	17.6	406.9	46.9	6	410.3	50.3	5	377.4	17.4	11
15	8	391.7	11.7	401.4	21.4	2	419.4	39.4	7	393.9	13.9	5
15	9	347.8	13.8	364.0	30.0	5	382.3	43.3	5	351.2	17.2	3
15	10	357.5	13.4	373.5	29.5	5	389.5	34.5	26	357.9	13.9	40
20	1	436.9	17.9	468.4	49.4	14	488.5	69.5	43	444.0	25.0	104
20	2	422.4	21.4	482.9	76.9	1,943	488.5	87.5	2,354	427.3	26.3	2,089
20	3	441.1	11.2	465.8	34.8	4	469.8	38.8	3	441.5	10.5	4
20	4	418.2	17.2	460.6	59.6	113	459.7	57.7	138	420.7	18.7	199
20	5	427.0	23.6	476.3	64.3	191	475.2	63.2	108	434.0	28.0	263
20	6	375.8	16.8	421.1	54.1	38	424.7	65.7	52	381.4	22.4	42
20	7	426.8	21.8	481.9	73.9	129	496.4	91.4	821	433.3	28.3	687
20	8	417.2	17.2	440.9	40.9	13	477.5	77.5	2,400	422.7	22.7	560
20	9	386.7	19.7	420.2	49.2	12	435.9	66.9	43	391.5	24.5	217
20	10	429.1	20.7	461.1	50.1	165	486.5	66.5	976	435.3	23.3	3,369
25	1	486.5	28.3	548.5*		4.2%	576.8*		7.3%	492.9*		1.9%
25	2	490.2	26.2	577.9*		6.5%	592.0*		7.2%	496.1	32.1	4,415
25	3	482.6	18.6	522.7	58.7	149	545.5	61.5	1,101	487.9	23.9	337
25	4	473.6	24.6	545.2*		0.9%	543.1*		2.2%	477.9	27.9	4,434
25	5	482.6	33.6	536.8*		6.8%	559.0*		11.2%	466.3*		4.2%
25	6	441.7	20.7	463.8	82.8	1,115	475.8	94.8	3,003	410.6	29.6	816
25	7	411.9	28.8	538.3*		2.4%	550.5*		5.2%	465.2*		1.7%
25	8	450.5	25.0	486.9	64.9	1,229	556.5*		11.7%	453.2*		1.4%
25	9	450.1	28.2	466.2	78.2	504	498.0*		5.9%	419.1*		2.8%
25	10	451.6	25.9	519.1*		6.1%	538.5*		8.4%	462.1*		4.2%
# instances (over 50)				44			42			44		
Avg. dev. 2phase vs. Xpolicy				−0.4%	−6.7%		−0.7%	−9.0%		−0.1%	−4.8%	
Avg. dev. 2phase vs. opt. policy				4.5%	141.5%		6.1%	193.2%		0.4%	11.7%	

Table 3 Solution Values and Computing Times for the Three Handling Policies by Considering the Branch-and-Cut Algorithm and $h = 0.5$

$ V_c $	Id	Two-phase		Policy 1			Policy 2			Policy 3		
		z^*	z_H^*	z^*	z_H^1	Seconds	z^*	z_H^2	Seconds	z^*	z_H^3	Seconds
5	1	302.5	7.5	305.0	10.0	0	317.0	22.0	0	302.5	7.5	0
5	2	267.5	7.5	273.5	13.5	0	272.0	12.0	0	267.5	7.5	0
5	3	283.0	4.0	288.5	9.5	0	284.5	5.5	0	283.0	4.0	0
5	4	255.5	3.5	262.0	10.0	0	263.0	11.0	0	255.5	3.5	0
5	5	209.5	13.5	204.5	8.5	1	218.0	22.0	0	202.5	6.5	0
5	6	189.0	13.0	192.5	16.5	0	199.0	23.0	0	189.0	13.0	0
5	7	244.0	11.0	253.0	20.0	0	250.0	17.0	0	244.0	11.0	0
5	8	195.5	4.5	198.5	7.5	0	195.5	4.5	0	194.0	3.0	0
5	9	282.5	6.5	285.0	9.0	0	295.0	11.0	0	282.5	6.5	0
5	10	196.5	9.5	198.5	11.5	0	204.0	17.0	0	196.0	6.0	0
10	1	362.5	29.5	396.5	36.5	5	421.0	88.0	3	372.0	39.0	4
10	2	346.5	26.5	394.5	74.5	3	402.0	82.0	2	351.0	31.0	1
10	3	353.5	15.5	374.5	36.5	2	381.5	43.5	1	353.5	15.5	0
10	4	320.5	21.5	344.0	33.0	4	338.0	39.0	5	324.0	25.0	11
10	5	344.5	29.5	376.0	61.0	3	396.0	81.0	4	346.5	31.5	6
10	6	276.0	31.0	327.5	64.5	9	340.5	95.5	8	283.5	38.5	4
10	7	323.5	38.5	374.0	88.0	2	380.5	94.5	5	326.0	40.0	7
10	8	341.5	26.5	355.0	40.0	1	387.5	49.5	7	343.5	28.5	2
10	9	312.5	29.5	339.0	56.0	1	376.5	93.5	4	317.5	34.5	1
10	10	299.0	22.0	327.0	50.0	3	357.5	80.5	4	300.0	23.0	2
15	1	423.5	56.5	495.0	92.0	116	529.0	162.0	1,564	440.0	67.0	2,699
15	2	433.0	65.0	543.0	155.0	1,751	581.5*		7.1%	439.0*		2.4%
15	3	431.0	31.0	477.5	77.5	19	501.5	85.5	24	432.0	32.0	7
15	4	413.0	49.0	474.0	73.0	533	474.0	73.0	5,849	417.5	52.5	6,910
15	5	452.0	62.0	550.0	154.0	1,280	562.0	166.0	1,312	468.0*		3.0%
15	6	385.5	50.5	470.0	106.0	408	476.5	141.5	477	393.5	58.5	173
15	7	448.0	88.0	580.0	166.0	1,236	578.4	169.4	2,970	442.0	80.0	5,041
15	8	438.5	58.5	487.0	66.0	86	513.5	100.5	1,705	447.5	34.5	3,933
15	9	403.0	69.0	471.0	132.0	70	555.5*		10.2%	412.0	73.0	2131
15	10	411.0	67.0	470.5	101.5	538	526.5*		2.0%	405.0*		3.1%
20	1	508.5	89.5	627.0*		6.6%	766.5*	35.5%		543.0*		15.3%
20	2	508.0	107.0	737.5*		26.0%	785.0*	37.9%		532.5*		20.8%
20	3	487.0	56.0	598.0*		4.9%	601.0*	6.2%		483.5	52.5	1,662
20	4	487.0	86.0	650.5*		14.9%	659.5*	27.7%		495.5*		11.3%
20	5	524.0	118.0	727.5*		26.5%	727.5*	28.0%		536.5*		18.0%
20	6	443.0	84.0	629.0*		22.8%	618.0*	27.7%		460.5*		10.2%
20	7	514.0	109.0	747.5*		23.1%	762.0*	33.1%		544.5*		21.2%
20	8	486.0	86.0	588.0*		8.6%	634.5*	23.3%		510.0*		15.8%
20	9	465.5	98.5	609.5*		13.4%	698.0*	35.0%		475.0*		11.5%
20	10	514.5	103.5	635.5*		17.3%	754.5*	35.3%		508.5*		15.7%
# instances (over 40)				30			27			28		
Avg. dev. 2phase vs. Xpolicy				−2.0%	−17.3%		−2.1%	−14.4%		−0.7%	−8.6%	
Avg. dev. 2phase vs. opt. policy				10.0%	84.7%		12.4%	131.1%		0.6%	0.0%	

ment in practice than an optimal policy. A higher h coefficient would linearly increase the corresponding handling cost incidence: if $h = 0.5$, the handling cost would become five times larger and, analogously, if $h = 1$ this cost would be ten times larger. In addition, Table 1 reports the relative importance of handling costs with respect to routing costs by varying the h coefficient, i.e., z_H^*/z_R . We do not report the computational time for any of the two-phase approaches, given that it is negligible.

Tables 2, 3, and 4 present the results obtained through the branch-and-cut algorithms summarized

in §4.3, with $h = 0.1$, $h = 0.5$, and $h = 1$, respectively. The optimal solution value z^* , the handling cost z_H and the computational time, in seconds, required for each policy and for each instance are reported. The time limit is 7,200 seconds and instances not solved to optimality within this limit are reported as follows. The best integer feasible solution found is marked with a an asterisk in the z^* column, and the percentage deviation between the best integer feasible solution and the best lower bound are reported in the following two columns. In the bottom lines, the number of instances solved to optimality are reported

Table 4 Solution Values and Computing Times for the Three Handling Policies by Considering the Branch-and-Cut Algorithm and $h = 1$

$ V_c $	Id	Two-phase		Policy 1			Policy 2			Policy 3		
		z^*	z_H^*	z^*	z_H^1	Seconds	z^*	z_H^2	Seconds	z^*	z_H^3	Seconds
5	1	310	15	315	20	0	339	44	0	310	15	0
5	2	275	15	287	27	0	284	24	0	275	15	0
5	3	287	8	298	19	0	290	11	0	287	8	0
5	4	259	7	272	20	0	274	22	0	259	7	0
5	5	223	27	213	17	0	240	44	0	209	13	0
5	6	202	26	207	27	0	222	46	1	201	21	0
5	7	255	22	273	40	0	267	34	0	255	22	0
5	8	200	9	206	15	0	200	9	0	197	6	0
5	9	289	13	294	18	0	306	22	0	289	13	0
5	10	206	19	207	15	0	221	31	0	202	12	0
10	1	392	59	433	73	4	509	176	15	411	78	26
10	2	373	53	469	149	3	484	164	6	382	62	5
10	3	369	31	411	73	5	407	32	3	369	31	1
10	4	342	43	371	48	3	368	55	18	347	37	42
10	5	374	59	437	122	7	468	107	17	378	63	46
10	6	307	62	389	108	30	436	191	42	322	77	71
10	7	362	77	460	162	7	475	189	29	366	80	60
10	8	368	53	393	64	2	434	61	9	372	57	17
10	9	342	59	395	112	3	457	146	10	352	69	6
10	10	321	44	375	88	12	437	150	15	323	46	17
15	1	480	113	587	184	199	667*	17.1%		507*	12.2%	
15	2	498	130	702*	3.9%		756*	27.2%		509*	17.1%	
15	3	462	62	535	85	369	565	76	574	464	64	364
15	4	462	98	539	118	379	546*	12.1%		457*	8.0%	
15	5	514	124	689*	6.9%		701*	5.8%		551*	19.5%	
15	6	436	101	559	140	5,713	618*	18.5%		452*	6.7%	
15	7	536	176	730	290	6,878	762*	14.4%		522*	20.0%	
15	8	497	117	553	132	160	614*	12.3%		482*	5.5%	
15	9	472	138	601	258	863	748*	33.3%		483*	14.2%	
15	10	478	134	572	203	3,112	681*	21.2%		455*	13.7%	
# instances (over 30)				28			21			21		
Avg. dev. 2phase vs. Xpolicy				−4.9%		−24.1%	−3.0%		−21.3%	−0.6%		−8.7%
Avg. dev. 2phase vs. opt. policy				13.3%		64.2%	16.2%		101.5%	0.5%		−1.9%

(line denoted as # instances (over W), where W is the number of instances considered) and, for each policy, the same policy two-phase approach and the optimal policy two-phase approach are compared (see Table 1).

The average improvement obtained by simultaneously optimizing routing and handling with respect to the corresponding policy two-phase approach is provided in the line denoted *Avg. Dev. 2phase vs. XPolicy*. This improvement is reported both in terms of the average percentage deviation between the solution values z^* and of the average percentage deviation of the handling costs z_H . Note that instances not solved to optimality have not been included in the average calculations. The improvement is smaller than 1% for $h = 0.1$ (see Table 2), but it can reach almost 5% with a larger h coefficient (i.e., $h = 1$ and Policy 1). However, the benefit obtained through the optimal policy, even with a two-phase approach, is still higher than the

benefit obtained through simultaneous optimization of routing and handling under a simplified policy. The line denoted as *Avg. Dev. 2phase vs. opt. policy* reports the average percentage deviation with respect to the optimal policy two-phase approach. The average percentage deviation between the solution values produced by these methods is quite large under the first two policies (up to 16.2% when applying the second policy with $h = 1$), but very limited under the third policy (0.4%, 0.6%, and 0.5% with $h = 0.1$, $h = 0.5$, and $h = 1$, respectively). In addition, the third policy produced some improving solutions with respect to the optimal policy two-phase approach. Note that instance difficulty increases with h .

Tables 5, 6, and 7 compare the standard branch-and-cut approach and the Benders-based algorithm in terms of solution quality, computational time and number of solved instances. We provide average results for the easy instances (solved by both meth-

Table 5 Solution Values and Computation Times for the Three Handling Policies by Considering the Branch-and-Cut Algorithm and the Benders-Based Branch-and-Cut Algorithm with $h = 0.1$

$ V_c $	Id	Policy 1			Policy 2			Policy 3		
		z_1^*	Sec.(B&C)	Sec.(Benders)	z_2^*	Sec.(B&C)	Sec.(Benders)	z_3^*	Sec.(B&C)	Sec.(Benders)
5	Avg.	—	0.1	0.1	—	0.0	0.0	—	0.0	0.1
10	Avg.	—	0.7	0.1	—	0.4	0.1	—	0.4	0.8
15	Avg.	—	5.3	1.9	—	7.4	2.6	—	13.9	205.1
20	1	—	14	3	—	43	15	—	104	6,900
20	2	—	1,943	762	—	2,354	389	—	2,089	3.1%
20	3	—	4	8	—	3	2	—	4	115
20	4	—	113	150	—	138	71	—	199	0.3%
20	5	—	191	98	—	108	50	—	263	4.1%
20	6	—	38	56	—	52	26	—	42	1,069
20	7	—	129	164	—	821	141	—	687	6.3%
20	8	—	13	10	—	2,400	506	—	560	3,628
20	9	—	12	16	—	43	27	—	217	2.0%
20	10	—	165	48	—	976	858	—	3,369	11.7%
25	1	—	4.2%	1.4%	—	7.3%	3.8%	—	1.9%	7.5%
25	2	—	6.5%	4.0%	—	7.2%	2.1%	—	4,415	10.2%
25	3	—	149	575	—	1,101	1,255	—	337	0.8%
25	4	545.2	0.9%	6,822	—	2.2%	1.2%	—	4,434	5.5%
25	5	—	6.8%	2.2%	—	11.2%	7.0%	—	4.2%	11.2%
25	6	—	1,115	758	—	3,003	445	—	816	5.0%
25	7	538.3	2.4%	3,550	550.5	5.2%	4,485	—	1.7%	9.2%
25	8	—	1,229	442	—	11.7%	8.3%	—	1.4%	7.9%
25	9	—	504	522	—	5.9%	2.2%	—	2.8%	12.7%
25	10	—	6.1%	4.4%	—	8.4%	4.1%	—	4.2%	9.1%
# instances (over 50)			44	46		42	43		44	34
Avg. seconds			331	213		737	253		103	1703
Avg. deviation			5.9%	3.0%		7.7%	4.1%		2.7%	9.6%

Table 6 Solution Values and Computation Times for the Three Handling Policies by Considering the Branch-and-Cut Algorithm and the Benders-Based Branch-and-Cut Algorithm with $h = 0.5$

$ V_c $	Id	Policy 1			Policy 2			Policy 3		
		z_1^*	Sec.(B&C)	Sec.(Benders)	z_2^*	Sec.(B&C)	Sec.(Benders)	z_3^*	Sec.(B&C)	Sec.(Benders)
5	Avg.	—	0.0	0.0	—	0.0	0.0	—	0.0	0.1
10	Avg.	—	3.3	0.5	—	4.3	0.6	—	3.8	5.6
15	1	—	116	29	—	1,564	81	—	2,699	6,031
15	2	—	1,751	627	581.5	7.1%	1,041	—	2.4%	7.7%
15	3	—	19	18	—	24	12	—	7	33
15	4	—	533	1,574	—	5,849	970	—	6,910	2.5%
15	5	—	1,280	454	—	1,312	1,313	—	3.0%	5.6%
15	6	—	408	200	—	477	58	—	173	594
15	7	—	1,236	4,033	—	2,970	336	—	5,041	7.8%
15	8	—	86	37	—	1,705	181	—	3,933	5,752
15	9	—	70	22	555.5	10.2%	1,461	—	2,131	3.3%
15	10	—	538	190	526.5	2.0%	3,075	—	3.1%	9.4%
20	1	627.0	6.6%	4,547	—	35.5%	17.9%	—	15.3%	22.6%
20	2	—	26.0%	16.1%	—	37.9%	11.0%	—	20.8%	32.1%
20	3	—	4.9%	8.1%	601.0	6.2%	4,946	—	1,662	6,741
20	4	—	14.9%	18.4%	—	27.7%	14.6%	—	11.3%	18.3%
20	5	—	26.5%	18.0%	—	28.0%	16.2%	—	18.0%	32.1%
20	6	—	22.8%	20.3%	—	27.7%	13.9%	—	10.2%	18.1%
20	7	—	23.1%	14.5%	—	33.1%	7.4%	—	21.2%	34.6%
20	8	—	8.6%	5.7%	—	23.3%	8.4%	—	15.8%	19.5%
20	9	—	13.4%	5.2%	—	35.0%	14.8%	—	11.5%	23.7%
20	10	—	17.3%	5.9%	749.5	35.3%	3,801	—	15.7%	29.6%
# instances (over 40)			30	31		27	32		28	25
Avg. seconds			202	240		516	110		340	768
Avg. deviation			17.5%	12.5%		31.8%	13.8%		12.4%	21.1%

Table 7 Solution Values and Computation Times for the Three Handling Policies by Considering the Branch-and-Cut Algorithm and the Benders-Based Branch-and-Cut Algorithm with $h = 1$

$ V_c $	Id	Policy 1			Policy 2			Policy 3		
		z_1^*	Sec.(B&C)	Sec.(Benders)	z_2^*	Sec.(B&C)	Sec.(Benders)	z_3^*	Sec.(B&C)	Sec.(Benders)
5	Avg.	—	0	0	—	0	0	—	0	0
10	Avg.	—	8	3	—	16	7	—	29	49
15	1	—	199	270	667	17.1%	3,738	—	12.2%	18.2%
15	2	—	3.9%	5.4%	—	27.2%	3.9%	—	17.1%	24.7%
15	3	—	369	227	—	574	211	—	364	577
15	4	—	379	4.9%	—	12.1%	3.7%	—	8.0%	14.0%
15	5	—	6.9%	6.1%	—	5.8%	10.6%	—	19.5%	22.2%
15	6	—	5,713	3,872	—	18.5%	7.7%	—	6.7%	10.0%
15	7	—	6,878	8.7%	747	14.4%	2,911	—	20.0%	26.8%
15	8	—	160	124	614	12.3%	2,762	—	5.5%	7.8%
15	9	—	863	596	—	33.3%	12.7%	—	14.2%	20.3%
15	10	—	3,112	1,156	—	21.2%	11.2%	—	13.7%	22.7%
# instances (over 30)			28	27		21	24		21	21
Avg. seconds			389	232		35	14		31	51
Avg. deviation			5.4%	5.7%		19.9%	8.4%		13.0%	18.5%

ods in less than five minutes), but we report more detailed results for the instances where the methods' performances differ substantially. The z^* column provides the optimal solution value for the instances solved to optimality with the Benders-based branch-and-cut algorithm and not solved to optimality with the standard branch-and-cut algorithm. The columns *sec.(B&C)* and *sec.(Benders)* report the computational time in seconds required if the instance has been solved to optimality within the 7,200 seconds time limit; otherwise they give the percentage deviation between the best known upper and lower bounds. The bottom line # instances (over W) reports the number of instances solved to optimality with respect to the number W of instances. The *Avg.seconds* line provides the average computational time in seconds for the instances solved by both methods, whereas *Avg.deviation* is the average percentage deviation for the instances neither method could solve to optimality. The Benders-based branch-and-cut approach improves both solution quality and computational time for the first two policies, allowing to solve to optimality 12 additional instances with respect to the standard branch-and-cut algorithm. The third policy seems more difficult to solve using the Benders-based branch-and-cut algorithm. This algorithm is probably weaker because of the linearization techniques adopted.

6. Conclusions

We have introduced the *traveling salesman problem with pickups, deliveries, and handling costs* (TSPPD-H), a single vehicle pickup and delivery problem in which handling costs are taken into account in the objective function and that allows for rearrangement of the

goods in the vehicle at customer locations. The handling policies available within the TSPPD-H framework offer more flexibility than do standard models and can yield important savings when routing costs are important with respect to handling costs.

The TSPPD-H was formulated as an integer linear program. Because this model involves a pseudopolynomial number of variables and is tractable only in the case of very small problem instances, we have derived three handling policies that considerably simplify the model and make good managerial sense. We have provided models applicable to each of these three policies and we have developed two exact algorithms for their resolution. The first is a classical branch-and-cut algorithm, whereas the second combines Benders decomposition and branch-and-cut.

Extensive computational experiments have shown that instances involving up to 25 customers can be solved optimally for all three policies and with either algorithm. For the first two policies, the Benders-based algorithm seems to outperform the pure branch-and-cut algorithm, whereas the reverse holds for the third policy. Our tests also show that the results obtained under Policy 1 are much worse than those obtained with Policy 3, or with Aslidis's algorithm, that is applying an optimal handling policy on an optimal route. No dominance relationship can be established between solving our model optimally with Policy 3 or applying Aslidis's algorithm.

Acknowledgments

The authors gratefully thank two anonymous referees for their valuable comments. The authors also thank Stefan Ropke, who provided the code for an implementation of the Edmonds-Karp algorithm for the maximum flow problem. This work was partly supported by the Canadian Natural Science and Engineering Research Council

under Grant 39682-05 and by Ministero dell'Istruzione, Università e Ricerca, Italy.

References

- Aslidis, A. 1989. Combinatorial algorithms for stacking problems. Unpublished doctoral dissertation, Massachusetts Institute of Technology, Cambridge.
- Aslidis, A. 1991. Minimization of overstockage in containership operations. H. E. Bradley, ed. *Operational Research'90*. Pergamon Press, Oxford, UK, 457–471.
- Benders, J. F. 1962. Partitioning procedures for solving mixed variables programming problems. *Numerische Mathematik* 4 238–252.
- Berbeglia, G., J.-F. Cordeau, I. Gribkovskaia, G. Laporte. 2007. Static pickup and delivery problems: A classification scheme and survey. *TOP: Official J. Spanish Soc. Stat. Oper. Res.* 15 1–31.
- Carrabs, F., R. Cerulli, J.-F. Cordeau. 2007a. An additive branch-and-bound algorithm for the pickup and delivery traveling salesman problem with LIFO or FIFO loading. *Inform. Systems Oper. Res.* 45 223–238.
- Carrabs, F., J.-F. Cordeau, G. Laporte. 2007b. Variable neighborhood search for the pickup and delivery traveling salesman problem with LIFO loading. *INFORMS J. Comput.* 19 618–632.
- Cordeau, J.-F., M. Iori, G. Laporte, J.-J. Salazar-González. 2009. A branch-and-cut algorithm for the pickup and delivery traveling salesman problem with LIFO loading. *Networks* 55 46–59.
- Edmonds, J., R. M. Karp. 1972. Theoretical improvements in algorithmic efficiency for network flow problems. *J. Assoc. Comput. Machinery* 19 248–264.
- Erdoğan, G., J.-F. Cordeau, G. Laporte. 2009. The pickup and delivery traveling salesman problem with first-in-first-out loading. *Comput. Oper. Res.* 36 1800–1808.
- Fischetti, M., P. Toth. 1997. A polyhedral approach to the asymmetric traveling salesman problem. *Management Sci.* 43(11) 1520–1536.
- Gendreau, M., G. Laporte, D. Vigo. 1999. Heuristics for the traveling salesman problem with pickup and delivery. *Comput. Oper. Res.* 26 699–714.
- Gribkovskaia, I., G. Laporte. 2008. One-to-many-to-one single vehicle pickup and delivery problems. B. Golden, S. Raghavan, E. Wasil, eds. *The Vehicle Routing Problem: Latest Advances and New Challenges*, Vol. 43. *Operations Research/Computer Science Interfaces Series*. Springer, New York, 359–377.
- Gribkovskaia, I., Ø. Halskau, G. Laporte, M. Vlček. 2007. General solutions to the single vehicle routing problem with pickups and deliveries. *Eur. J. Oper. Res.* 180 568–584.
- Halskau, Ø., A. Løkketangen. 1998. Analyse av distribusjonsopplegget ved Sylte Mineralvannfabrikk A/S. Technical Report M9812, Møreforskning, Molde University College, Molde, Norway.
- Hoff, A., A. Løkketangen. 2006. Creating lasso-solutions for the traveling salesman problem with pickup and delivery by tabu search. *Central Eur. J. Oper. Res.* 14 125–140.
- Hoff, A., I. Gribkovskaia, G. Laporte, A. Løkketangen. 2009. Lasso solution strategies for the vehicle routing problem with pickups and deliveries. *Eur. J. Oper. Res.* 192 755–766.
- Ladany, S. P., A. Mehrez. 1984. Optimal routing of a single vehicle with loading and unloading constraints. *Transportation Planning Tech.* 8 301–306.
- Magnanti, T. L., R. T. Wong. 1981. Accelerating Benders decomposition: Algorithmic enhancements and model selection criteria. *Oper. Res.* 29(3) 464–484.
- Naddef, D., S. Thienel. 2002. Efficient separation routines for the symmetric traveling salesman problem. I: General tools and comb separation. *Math. Programming* 92 237–255.
- Pacheco, J. A. 1997. Heurístico para los problemas de rutas con carga y descarga en sistemas LIFO. *Qüestió* 21 153–175.
- Padberg, M. W., M. Grötschel. 1985. Polyhedral computations. E. L. Lawler, J. K. Lenstra, A. H. G. Rinnooy Kan, D. B. Shmoys, eds. *The Traveling Salesman Problem*. Wiley, Chichester, UK, 307–360.
- Petersen, H. L., O. B. G. Madsen. 2009. The double travelling salesman problem with multiple stacks—Formulation and heuristic solution approaches. *Eur. J. Oper. Res.* 198 139–147.
- Petersen, H. L., C. Archetti, M. G. Speranza. 2010. Exact solutions to the double travelling salesman problem with multiple stacks. *Networks*, ePub online February 19, 2010.